

Entregable 2

Conocimientos previos

1.1. Stacks: Un stack o pila es una lista ordinal o estructura de datos en la que el modo de acceso a sus elementos es de tipo LIFO (del inglés Last In First Out, el último en entrar, primero en salir) que permite almacenar y recuperar datos. Se aplica en multitud de ocasiones informática debido a su simplicidad y ordenación implícita de la propia estructura.

Para poder usar el contenedor stack debemos añadir a la cabecera de nuestro programa `#include <stack>`.

En este ejercicio, se recomienda usar stacks para ir ordenando los puntos que formarán parte del perímetro del convexhull.

1.2. Operaciones básicas de un stack o pila

- Para el manejo de datos se cuenta con dos operaciones básicas: apilar (**push**), que coloca un objeto en la pila, y su operación inversa: retirar (**pop**), que saca el último elemento apilado.
- El acceso a los elementos de la pila es siempre desde la parte superior de ésta, es decir, al último objeto apilado. La operación que permite la obtención de este elemento sin retirarlo de la lista es **top**.
- Para conocer el tamaño de la pila podremos emplear el método **size** que retorna un entero y para conocer si la pila está llena o vacía podemos llamar a los métodos **empty** y **full** que retornan un valor booleano.

2.1. Vectores: Esta clase genérica permite almacenar una colección de objetos del mismo tipo. Al igual en listas o arrays, podemos acceder a los elementos del contenedor vector usando un índice y el operador de indexación (`[i]`), y además estos elementos ocupan posiciones contiguas de memoria.

Para poder usar el contenedor vector debemos añadir a la cabecera de nuestro programa `#include <vector>`.

Dado que implementa un template, debemos indicar en nuestro programa el tipo de dato de los elementos de la colección. Esto lo hacemos mediante la siguiente sintaxis: `std::vector<T>` identificador;

Ejemplo de inicialización de un vector de 5 elementos de tipo struct Point: `std::vector<Point> puntos(5);`

2.2. Operaciones básicas de un vector

- Para conocer el tamaño de un vector podremos emplear el método **size** que retorna un entero y para conocer si el vector está lleno podemos llamar al método **empty** que retorna un valor booleano.
- En cuanto al acceso a los elementos, los métodos **back** y **front** devuelven la referencia al último y primer elemento de la lista respectivamente. En caso de querer insertar un elemento en la última posición de la lista utilizamos el método **push_back**.
- Para iterar por el vector desde el inicio hasta el final podemos utilizar los métodos **begin** y **end**, y en caso de que deseemos iterar en sentido reverso podemos utilizar **rbegin** y **rend**.

3. Librería SFML para Visual Studio

Simple and Fast Multimedia Library (SFML) es una [biblioteca](#) de desarrollo de software [multiplataforma](#) diseñada para proveer una simple [interfaz de programación](#) para diversos elementos multimedia en computadoras. Está escrita en [C++](#), con versiones homólogas disponibles

en [Ada](#), [C](#), [Crystal](#), [D](#), [Euphoria](#), [Go](#), [Java](#), [Julia](#), [.NET](#), [Nim](#), [OCaml](#), [Python](#), [Ruby](#) y [Rust](#).^[3] Se han desarrollado versiones experimentales para [Android](#) y [iOS](#) con el lanzamiento de SFML 2.2.^[4]

Fuente: Wikipedia

[Link para descarga y pasos de instalación.](#)

Instalación de SFML descargando el paquete estable más actualizado v3.0.0 de 64-bits. Extraer de la carpeta comprimida la carpeta "SFML-3.0.0" y colocarlo en la raíz del equipo. A continuación, edita las propiedades del proyecto para todas las configuraciones de la plataforma x64.

Primero, indica en bibliotecas adicionales de C++ la carpeta include del SDK, y en las bibliotecas adicionales del vinculador (linker) la carpeta lib del SDK.

Una vez completado, indicar en las entradas del vinculador las librerías "sfml-graphics.lib", "sfml-system.lib", "sfml-window.lib" y "sfml-network.lib" en las dependencias adicionales.

Por último, incluye los archivos dlls de la carpeta de binarios en la carpeta de tu proyecto de visual studio.

A continuación, tenéis un código para poner en el archivo main y probar si las librerías se han cargado y configurado correctamente:

```
#include <SFML/Graphics.hpp>

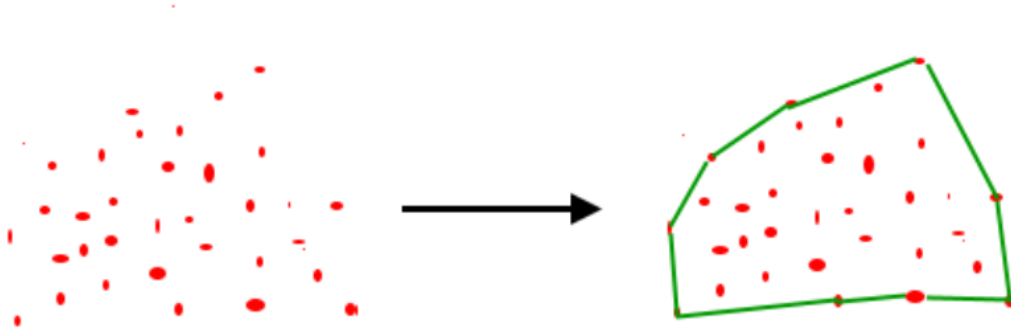
int main(){
    sf::RenderWindow window(sf::VideoMode({ 200, 200 }), "SFML works!");
    sf::CircleShape shape(100.f);
    shape.setFillColor(sf::Color::Green);

    while (window.isOpen())
    {
        while (const std::optional event = window.pollEvent())
        {
            if (event->is<sf::Event::Closed>())
                window.close();
        }

        window.clear();
        window.draw(shape);
        window.display();
    }
}
```

Convex hull

El convexhull o envolvente convexa es un algoritmo que, dados los puntos de un objeto en el espacio, permite crear un polígono que encierra a todos los puntos de dicha nube. Existen varios algoritmos importantes para resolver este problema: Quickhull, Jarvis y Graham.



Podéis encontrar una descripción del algoritmo en la Wikipedia, es recomendable leerlo tanto en inglés como en español. [Link](#).

El objetivo es implementar el algoritmo y dibujar con la librería SFML los puntos del conjunto y la envolvente. Recordad que en la página oficial de la librería [SFML](#) podéis encontrar información y tutoriales, recordad usar la versión 3.0.0, hay variaciones con respecto a las anteriores.

En la página siguiente hay una guía de implementación de otros años. No es necesario emplear una pila para su implementación.

Nosotros nos centraremos en el algoritmo de Graham que consta de dos fases:

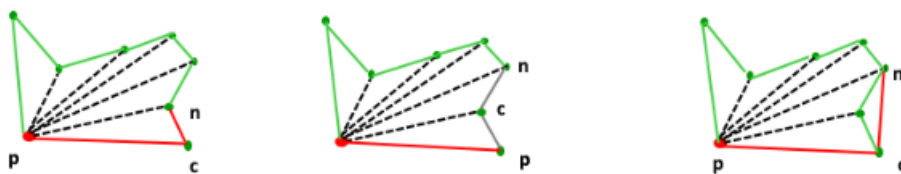
1. **Algoritmo de ordenación propio** → Se ordena los puntos de mayor a menor de acuerdo con el eje y, si dos puntos comparten la misma coordenada entonces se ordenan de acuerdo con el eje x.
2. **Algoritmo de orientación** → Evaluar si una tripleta de puntos (p, q, r) se orienta en sentido horario o antihorario. Con la siguiente fórmula: $(q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y)$ se puede concluir que si toma valor 0 los puntos son colineales, si toma valor positivo rota en sentido de las agujas del reloj y si toma valor negativo toma sentido antihorario.
3. **Algoritmo Convexhull – Graham** → El primer punto es el que se encuentre en la esquina inferior izquierda y se va recorriendo la lista tomando 3 puntos en todo momento, uno inicial, uno medio y uno final. se hace un recorrido de la lista (conocido como el scan de Graham) en el que se emplean áreas signadas para decidir si un punto debe ser eliminado o no. Si los puntos están ordenados en sentido antihorario, el área signada de los puntos P_{i-1} , P_i , P_{i+1} nos dice si el punto P_i debe ser eliminado. En efecto, si el área no es positiva, el punto P_i es interior al triángulo P_{i-1} , O, P_{i+1} , donde O es el centro de la ordenación, y, por tanto, interior al cierre convexo. En este caso debe ser eliminado. Para que esta segunda fase proporcione la lista correcta de vértices del cierre convexo es preciso incluir un detalle más: cada vez que se elimine un punto de la lista, se debe dar un paso atrás en el recorrido, ya que al eliminar un punto puede suceder que los tres que pasan a ser consecutivos den lugar a un área signada no positiva, en cuyo caso habría que eliminar el punto intermedio de nuevo.

El número total de estas operaciones que se realizan es del orden de $O(n \log n)$.

PASOS:

Sean los puntos $[0... n-1]$ del array de puntos de entrada.

- 1) Busca el punto más bajo comparando las coordenadas y de todos los puntos. Si hay dos puntos con el mismo valor y, entonces se considera el punto con menor valor de coordenada x. El punto más bajo es P_0 . Pon P_0 en la primera posición del stack.
- 2) Considerar los $n-1$ puntos restantes y ordenarlos por ángulo polar en sentido antihorario alrededor de puntos[0]. Si el ángulo polar de dos puntos es el mismo, pon primero el punto más cercano.
- 3) Una vez ordenados, comprueba si dos o más puntos tienen el mismo ángulo. Si dos o más puntos tienen el mismo ángulo, entonces elimina todos los puntos con el mismo ángulo excepto el punto más alejado de P_0 . Añade dicho punto al stack de tamaño m.



- 4) Si el tamaño del array es menor que 3, devolver.
- 5) Crear una pila vacía 'S' y empujar los puntos[0], puntos[1] y puntos[2] a S.
- 6) Procesa los $m-3$ puntos restantes uno a uno. Hacer lo siguiente para cada punto 'points[i]'
 - 6.1) Seguir quitando puntos de la pila mientras la orientación de los 3 puntos siguientes no sea antihoraria (o no giren a la izquierda).
 - a) Punto situado junto al superior de la pila
 - b) Punto en la parte superior de la pila
 - c) puntos[i]
 - 6.2) Empuja puntos[i] a S